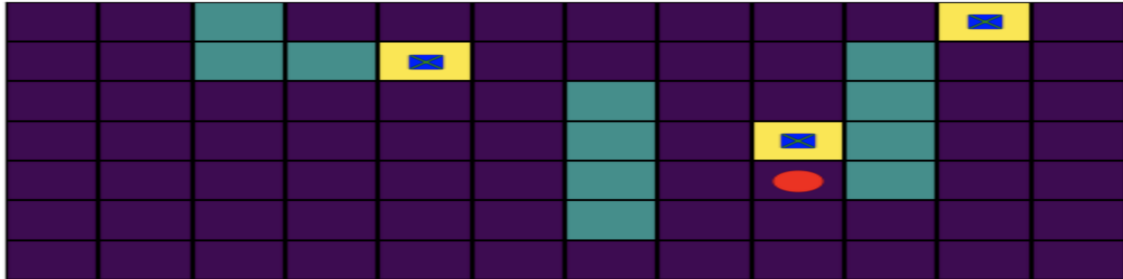


SOKOBAN AI

Name: AbdulKader Gobena Denboba

Group: 332CC



1. **Introduction** – Implementation sturcture.
2. **Heuristic Implementation** – hearaustic design
3. **LRTA *** – Irta* algorithm Implementation
4. **Simulated Annealing** – Simulated Annealing algorithm Implementation
5. **Analysis & Discussion** – Performance comparison with charts.
6. **Optimisation and summary**
7. **Testing Guide** – running and and testing the algorithms.

1. Introduction

My Sokoban AI implementation consists of three core components:

- **Irta_star.py**: Learning Real-Time A* algorithm implementation
- **simulated_annealing.py**: Simulated Annealing optimization approach
- **heuristic.py**: Shared heuristic functions for both algorithms

The implementation and results of these approaches are as follows

2. Heuristic Implementation

2.1 Manhattan Distance Calculation

```
def manhattan_distance(pos1: Tuple[int, int], pos2:
Tuple[int, int]) -> int:
    return abs(pos1[0] - pos2[0]) + abs(pos1[1] -
pos2[1])
```

- Uses the sum of absolute differences in x and y coordinates
- I chose it over Euclidean distance because:
 - Sokoban movement is strictly **grid-based** (4-directional)
 - More accurately represents actual movement costs
 - Computationally ***simpler*** (no square root operations)

2.2 Path Existence Check

```
def path_exists(start: Tuple[int, int], end:
Tuple[int, int], obstacles: Set[Tuple[int, int]],
max_depth: int = 20) -> bool:
    ...
```

Algorithm Characteristics:

- Breadth-First Search (BFS) implementation
- Depth-limited to prevent excessive computation
- Obstacle-aware (avoids walls and boxes)

Optimizations:

- Early termination when target is found
- Visited set prevents redundant checks
- Configurable depth limit balances completeness vs performance

Usage Context:

- Verifies if boxes can be pushed to targets
- Checks player movement possibilities
- Helps identify potential deadlocks

2.3 Minimum Matching Distance

```
def min_matching_distance(boxes: List[Tuple[int,
int]], targets: List[Tuple[int, int]]) -> float:
    ...
```

Algorithm Evolution:

1. Initially, I used Greedy Approach for:
 - Simple implementation
 - Suboptimal for complex configurations
 - Faster for small instances
2. Finally, I decided on Hungarian Algorithm for:
 - Optimal assignment guarantee
 - Implemented via SciPy's `linear_sum_assignment`
 - Better for complex puzzles

Implementation Details:

- Creates cost matrix of box-to-target distances
- Solves the assignment problem
- Returns sum of optimal distances

2.4 Deadlock Heuristic

```
def deadlock_heuristic(state) -> float:
    penalty = 0
    obstacles = state.obstacles
    box_positions = state.positions_of_boxes
    targets = state.targets
    ...
```

Deadlock Detection Methods:

1. **Basic Blockage:**
 - Checks horizontal and vertical wall blockages

- Identifies simple deadlocks
- 2. **Corner Analysis:**
 - Detects boxes pushed against two perpendicular walls
 - Special cases for corner positions
- 3. **Dynamic Evaluation:**
 - Differentiates between permanent and temporary deadlocks
 - Penalty system

Penalty Points:

- **1000 points:** For complete irreversible deadlock
- **500 points:** For potentially recoverable deadlock
- **100 points:** For partial blockage warning

2.5 Sokoban Heuristic

```
def sokoban_heuristic(state) -> float:  
    ...
```

Composite Heuristic Components:

1. **Base Distance Metric:**
 - Optimal box-to-target assignment cost
 - Weighted by path feasibility
2. **Player Positioning:**
 - Distance to nearest movable box
 - Average distance to all boxes
3. **State Quality Evaluation:**
 - Deadlock potential assessment
 - Adjacent box configuration analysis
 - Movement complexity factor
 - First, Second and diagonal neighbors aware

Optimization Techniques:

- Path existence caching

- Matrix operations for distance calculations
- Progressive penalty scaling based on the game progress

3. LRTA* Implementation

3.1 Algorithm Overview

Learning Real-Time A* (LRTA*) implementation provides an online search solution for Sokoban that combines heuristic search with learning capabilities. It maintains and updates heuristic estimates during the search process.

Key Features:

- **Real-time decision making** - Chooses moves without complete lookahead
- **Heuristic learning** - Improves estimates during execution
- **Adaptive exploration** - Dynamically adjusts search parameters
- **Plateau handling** - Implements specialized backtracking for **local optima**

3.2 Core Implementation Components

3.2.1 State Representation

```
def get_state_key(self, state) -> str:
    box_positions = frozenset((box.x, box.y) for box
in state.bboxes.values())
    return
f"{state.player.x},{state.player.y}|{hash(box_positio
ns)}"
```

- **Compact state encoding** combining player position and box configuration
- Uses frozenset for box positions to ensure consistent hashing

- Efficient string representation for dictionary lookups

3.2.2 Heuristic Management

```
def get_heuristic(self, state) -> float:
    key = self.get_state_key(state)
    if key not in self.h_values:
        self.h_values[key] = sokoban_heuristic(state)
    return self.h_values[key]

def update_heuristic(self, state, value: float):
    key = self.get_state_key(state)
    self.h_values[key] = max(value,
self.h_values.get(key, 0))
```

- **Lazy initialization** of heuristic values
- **Caching mechanism** to avoid redundant calculations
- **Conservative updates** using max() to maintain **admissible estimates**

3.3 Search Algorithm

3.3.1 Main Solving Loop

```
def solve(self, initial_state) ->
Optional[List[int]]:
    current_state = initial_state
    path: List[int] = []
    best_path = None
    best_h_value = float('inf')
    iterations = 0
    plateau_count = 0
    visited_states: Set[str] = set()

    ...
```

- **Iterative deepening** approach with iteration limit
- **Best-path tracking** for solution recovery
- **Plateau detection** mechanism to escape local optima

3.3.2 Adaptive Visit Control

```
...
adaptive_visits = self.max_visits * (2 +
progress_ratio * 4 + len(current_state.targets) / 3)
if progress_ratio > 0.7:
    adaptive_visits *= 2.5
elif progress_ratio > 0.5:
    adaptive_visits *= 1.8
elif boxes_on_target == 0:
    adaptive_visits *= 0.7
...
```

- **Dynamic visit limits** based on puzzle progress
- **Non-linear scaling** for different solution phases
- **Target count consideration** for puzzle complexity

3.3.3 Backtracking Mechanism

```
...
backtrack_depth = min(max(int(len(path) * 0.2), 5),
25)
for _ in range(backtrack_depth):
    if path:
        old_key = self.get_state_key(current_state)
        visited_states.discard(old_key)
        self.visit_counts[old_key] = 0
        path.pop()
...
```

- **Proportional backtracking** based on path length
- **State cleanup** of visited sets and counts
- **Bounded depth** to prevent excessive backtracking

3.4 Move Selection Strategy

3.4.1 Priority Calculation

```
...
base_cost = 2 if move >= BOX_LEFT else 1
h_next = self.get_heuristic(next_state)
visit_penalty = 0.3 * self.visit_counts[next_key] *
(1.5 if iterations > self.max_iterations * 0.5 else
1)
delta_target = sum(1 for b in
next_state.positions_of_boxes if b in
next_state.targets) - boxes_on_target
priority = base_cost + h_next + visit_penalty - 8 *
delta_target if delta_target > 0 else base_cost +
h_next + visit_penalty + 10 * abs(delta_target)
...
```

- **Multi-factor evaluation** combining:
 - Base movement cost
 - Heuristic estimate
 - Visit frequency penalty
 - Target achievement bonus
- **rewards** for progress vs regression

3.4.2 Deadlock Awareness

```
...
if move >= BOX_LEFT:
    moved_box = next((p for p in
next_state.positions_of_boxes if p not in
current_state.positions_of_boxes), None)
    if moved_box:
        x, y = moved_box
        if sum((x + dx, y + dy) in
```

```

next_state.obstacles for dx, dy in [(0,1), (0,-1), (1,0), (-1,0)] >= 2 and moved_box not in
next_state.targets:
    priority += 5
...

```

- **Corner analysis** for potential traps
- **Penalty application** to discourage risky moves

3.5 Memory Management

3.5.1 State Visitation Tracking

```

...
visited_states.add(key)
self.visit_counts[key] += 1
...

```

- **Duplicate prevention** through visited set
- **Visit counting** for adaptive penalties

3.5.2 Periodic Cleanup

```

...
if iterations % 30 == 0 and len(visited_states) > 1000:
    try:
        temp_state = initial_state.copy()
        recent_keys = set()
        for move in path[-20:]:
            temp_state.apply_move(move)

        recent_keys.add(self.get_state_key(temp_state))

    visited_states.intersection_update(recent_keys)
    except ValueError:
        visited_states = {key}
...

```

- **Memory optimization** through periodic pruning

- **Focus on recent path** for relevance
- **Exception handling** for state consistency

3.6 Solution Verification

```
...
def verify_solution(self, initial_state, moves:
List[int]) -> bool:
    if not moves:
        return False
    state = initial_state.copy()
    try:
        for move in moves:
            state.apply_move(move)
        return state.is_solved()
    except ValueError:
        return False
...
```

- **Complete replay** of solution moves
- **State validation** through copy and apply
- **Error handling** for illegal moves
- **Termination check** using game rules

3.7 Performance Considerations

- **Iteration limits** prevent infinite execution
- **Adaptive parameters** respond to search progress
- **Balanced exploration** through visit counting
- **Efficient data structures** for state management
- **Incremental learning** through heuristic updates

4. Simulated Annealing Implementation

4.1 Algorithm Overview

The Simulated Annealing implementation provides a stochastic optimization approach for solving Sokoban puzzles. It uses an initial temperature and a restart temperature to efficiently backtrack and manage the Sokoban game, incorporating a plateau limit to handle local optima.

Key Features:

- **Temperature-based exploration** - Balances exploration and exploitation
- **Plateau escape mechanisms** - Handles stuck conditions effectively
- **Adaptive restart strategy** - Resets search when progress stalls
- **State visitation tracking** - Avoids redundant exploration

4.2 Core Implementation Components

4.2.1 Parameter Configuration

```
def __init__(self,
    initial_temp: float = 75.0,
    cooling_rate: float = 0.995,
    min_temp: float = 0.5,
    max_iterations: int = 50000,
    restart_temp: float = 30.0,
    plateau_limit: int = 50):
    ...
```

- **Initial temperature:** Controls early exploration intensity (75.0)
- **Cooling rate:** Geometric temperature reduction (0.995)
- **Minimum temperature:** Lower bound for cooling (0.5)
- **Iteration limit:** default Maximum search steps (50,000)
- **Restart temperature:** Reset temperature for plateaus (30.0)
- **Plateau limit:** Iterations before considering restart (50)

4.2.2 State Representation

```
def get_state_key(self, state) -> str:
    box_positions = sorted((b.x, b.y) for b in
state.bboxes.values())
    return
f"p{state.player.x},{state.player.y}|b{box_positions}
"
```

- **Canonical representation** using sorted box positions
- **Player position** included in state key
- **String format** enables efficient dictionary lookups

4.3 Search Algorithm

4.3.1 Main Solving Loop

```
def solve(self, initial_state) ->
Optional[List[int]]:
    current_state = initial_state
    current_path = []
    best_state = current_state
    best_path = []
    best_score = sokoban_heuristic(current_state)
    temperature = self.initial_temp
    iterations = 0
    last_improvement = 0
    visited_states = {}

    ...
```

- **Smart preservation**: maintains best solution found

- **Progress tracking** : through iteration counting
- **Temperature management**: controls search behavior
- **State visitation tracking**: prevents cycles

4.3.2 Move Selection Strategy

```
...
sampled_moves = random.sample(possible_moves, min(5,
len(possible_moves)))

move_scores = []
for move in sampled_moves:
    next_state = current_state.copy()
    next_state.apply_move(move)
    state_key = self.get_state_key(next_state)
    visits = visited_states.get(state_key, 0)
    if visits > 2:
        continue
    score = (-10 if move >= 4 else 0) + visits * 5
    move_scores.append((move, next_state, state_key,
score))

...
```

- **Diverse sampling** considers multiple moves
- **Visit-based filtering** avoids over-visited states
- **Scoring system**:
 - Prefers box pushes (moves ≥ 4)
 - Penalizes frequently visited states
- **Balanced evaluation** combines multiple factors

4.3.3 Temperature-Driven Decision Making

```
...
if random.random() < temperature / self.initial_temp:
    move, next_state, state_key, _ =
random.choice(move_scores)
else:
    move, next_state, state_key, _ = min(move_scores,
key=lambda x: x[3])

...
```

- **Exploration phase:** Random move selection (high temp)
- **Exploitation phase:** Greedy selection (low temp)
- **Smooth transition** based on temperature ratio

4.4 Solution Quality Evaluation

4.4.1 Acceptance Criteria

```
...
if next_cost == 0 or next_state.is_solved():
    return current_path + [move]

if next_cost < best_score:
    best_state, best_path, best_score = next_state,
    current_path + [move], next_cost
    last_improvement = iterations
...
```

- **Direct termination** on perfect solution
- **Updating** when improving solutions found
- **Progress tracking** for plateau detection

4.4.2 Probabilistic Acceptance

```
...
def acceptance_probability(self, old_cost: float,
    new_cost: float, temperature: float) -> float:
    return 1.0 if new_cost < old_cost else
    math.exp((old_cost - new_cost) / temperature)
...
```

- **Deterministic acceptance** of improving moves
- **Probabilistic acceptance** of worse moves:
 - Higher probability at high temperatures
 - Considers cost degradation magnitude
 - Boltzmann distribution for energy states

4.5 Plateau Handling Mechanisms

4.5.1 Adaptive Restart

```
...
if iterations - last_improvement >
self.plateau_limit:
    if random.random() < 0.5:
        current_state, current_path = best_state,
best_path[:]
        temperature = self.restart_temp
    else:
        for _ in range(random.randint(10, 15)):
            moves =
current_state.filter_possible_moves()
            if not moves:
                break
            move = random.choice([m for m in moves if
m >= 4] or moves)
            current_state.apply_move(move)
            current_path.append(move)
            temperature = self.initial_temp
        visited_states.clear()
        last_improvement = iterations
    continue
...
```

- **Dual-strategy approach:**
 1. Reset to best solution (50% chance)
 2. Random walk (10-15 moves, 50% chance)
- **Temperature reset** to intermediate value
- **State cache clearing** for fresh exploration
- **Preference for box pushes** during random walk

4.5.2 Temperature Management

```
...
temperature *= self.cooling_rate
if temperature < self.min_temp:
    temperature = self.restart_temp
```

...

Restart mechanism prevents freezing

4.6 Performance Considerations

- **Heuristic caching** avoids redundant calculations
- **Balanced move sampling** between exploration/exploitation
- **Adaptive parameter control** responds to search progress
- **Efficient state tracking** through compact keys
- **Controlled randomness** for reproducible behavior

4.7 Termination Conditions

- **Solution found** (heuristic = 0 or is_solved() = True)
- **Iteration limit reached** (max_iterations)
- **No possible moves** (filter_possible_moves() empty)
- **Resource exhaustion** (time/memory constraints)

The implementation is an effective handling of Sokoban's complex search space through careful balance of probabilistic exploration and heuristic-guided optimization. The temperature schedule and restart mechanisms work together to escape local optima while progressively focusing on promising solution regions

5. Analysis and Discussion

5.1 Performance Comparison

After testing both algorithms on 9 different Sokoban maps of varying complexity, here's a detailed analysis:

Key Observations:

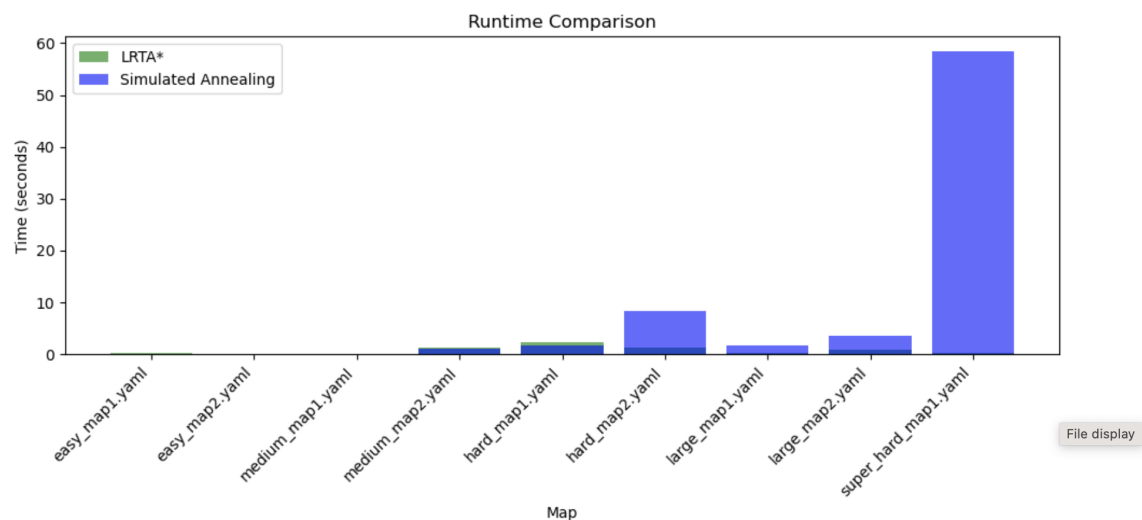
1. **LRTA*** works faster on for all maps
2. **Simulated Annealing** is almost as fast as Lrta except for supper_hard_map1.

3. **Both** use nearly similar numbers of pull moves except for hard_map1 hard_map2 and medium_map2
4. **LRTA*** depends on exploring states during search

5.2 Results Overview

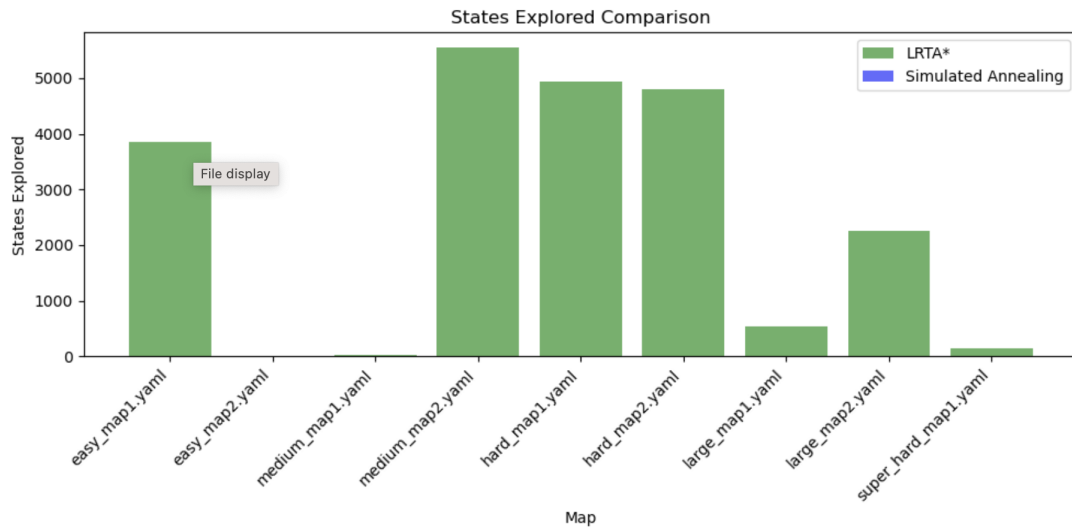
Time Performance

- LRTA* is faster for all map complexity
- Simulated Annealing takes longer for hard maps especially for super hard
- Time grows steadily for LRTA*, while SA varies more



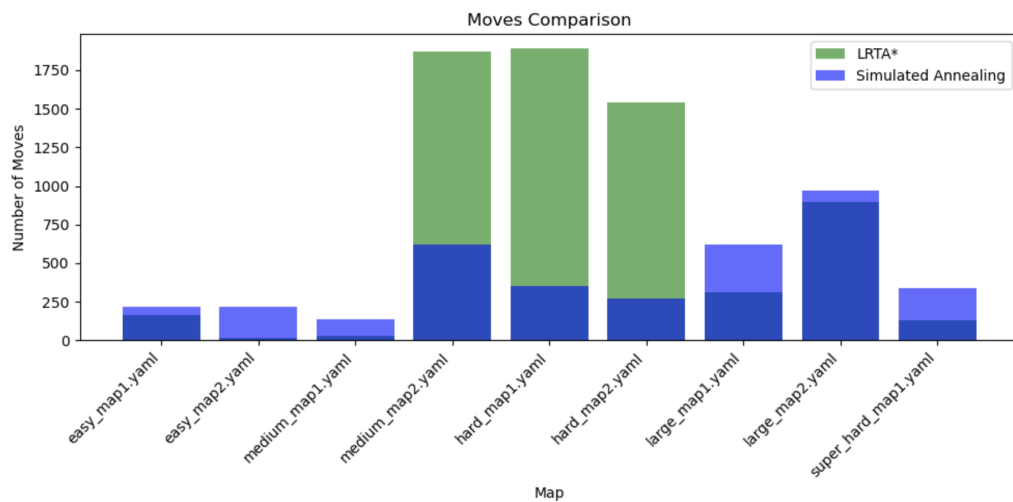
States Explored

- LRTA* tracks all explored states
- Simulated Annealing has no explored states
- More states explored usually means longer solutions



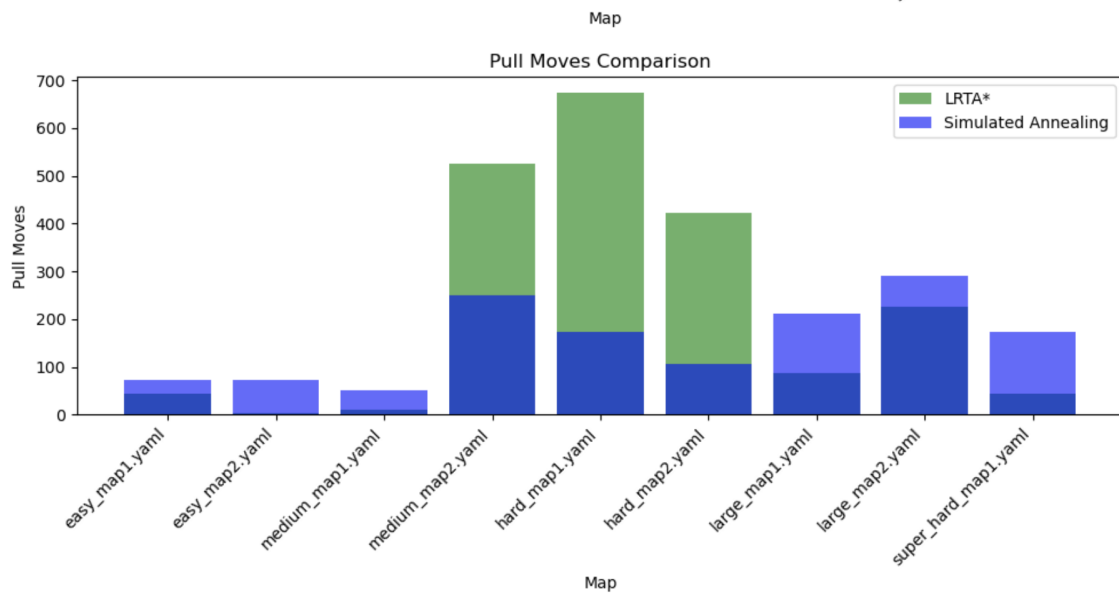
Moves

- **LRTA*** varies more between maps and uses more moves for medium_map_2 and hard_maps
- **SA** uses less moves compared to **LRTA***



Pull Moves

- Both use almost pull moves on average
- LRTA* varies more between maps using more moves for hard_maps and medium_map 1



6. Summary

1. While both algorithms use identical heuristics, LRTA*'s state caching mechanism and state exploring enables efficient reuse of previously computed values, making it the faster algorithm.
2. LRTA*'s learning mechanism allows it to improve path selection over time, while SA's temperature-based acceptance can sometimes lead to suboptimal path choices.
3. LRTA*'s learning mechanism and heuristic guidance provide more consistent performance, while SA's randomness can lead to varying results.
4. SA benefits significantly from restarting and its the heuristic system, which detects deadlocks with dynamic penalties.
5. Both algorithms benefit from plateau handling and restarting mechanisms, each implementing these features according to their specific conditions.

Optimization

Both algorithms significantly benefit from key optimizations including:

- Visit frequency penalties (avoiding redundant states)
- Progress-based rewards and achievement incentives
- Risk/danger penalties (for deadlocks or unstable moves)
- Backtracking with incremental learning
- Heuristic caching (eliminating redundant calculations)
- Adaptive restart strategies
- Efficient data structure implementation

These enhancements collectively improve efficiency, solution quality, and convergence speed.

7. Code Execution Guide

7.1 Quick Start

7.1.1 Command Line Execution

Test both algorithm with all maps

```
python main.py
```

```
(.venv) abdi@mac:~/PycharmProjects/AI_PROJECTS$ python main.py

=== Testing LRTA* Algorithm ===

Testing easy_map1.yaml...
Solution found with 165 moves
States explored: 3863
Pull moves used: 45
Time taken: 0.59 seconds

Solution visualization saved in tests/solutions/lrta*/easy_map1
```

- **Automatically** runs all test cases
- Displays **text-based results** in terminal
- Saves solutions

Test each algorithm with all the test cases

python main.py -a [algorithm]

```
(.venv) abdi@mac:~/PycharmProjects/AI_PROJECTS$ python main.py simulated-annealing

=== Running SIMULATED-ANNEALING Algorithm ===

Testing easy_map1.yaml...
Solution found with 343 moves
States explored: 0
Pull moves used: 89
Time taken: 0.44 seconds

Solution visualization saved in tests/solutions/simulated-annealing/easy_map1

Testing easy_map2.yaml...
```

- **Automatically** runs all test cases on the algorithm
- Displays **text-based results** in terminal
- Saves solutions

Test each algorithm with 1 map at a time

python main.py -a [algorithm] -i [map_file]

```
(.venv) abdi@mac:~/PycharmProjects/AI_PROJECTS$ python main.py lrt* tests/easy_map1.yaml

=== Running LRTA* Algorithm ===
Solution found with 165 moves
States explored: 3863
Pull moves used: 45
Time taken: 0.60 seconds

Solution visualization saved in tests/solutions/lrt*/easy_map1
(.venv) abdi@mac:~/PycharmProjects/AI_PROJECTS$
```

Or

```
(.venv) abdi@mac:~/PycharmProjects/AI_PROJECTS$ python main.py -a lrt* -i tests/easy_map1.yaml

=== Running LRTA* Algorithm ===
Solution found with 165 moves
States explored: 3863
Pull moves used: 45
Time taken: 0.60 seconds

Solution visualization saved in tests/solutions/lrt*/easy_map1
(.venv) abdi@mac:~/PycharmProjects/AI_PROJECTS$
```

- Tests the algorithm with the test
- Displays **text-based results** in terminal
- Saves solutions

Solution

- The solution will be always saved in:

tests/solutions/[algorithm_name]/[map_name]/

- Includes initial and final map states
-

7.2 More visulation with video and gif.

Note: this needs more time to proceess all the photos and generate the gif file

- write any of the above commands with -g ate the end.

```
(.venv) abdi@mac:~/PycharmProjects/AI_PROJECTS$ python main.py -a lrta* -i tests/hard_map2.yaml -g

=== Running LRTA* Algorithm ===
Solution found with 1540 moves
States explored: 4803
Pull moves used: 422
Time taken: 1.66 seconds
GIF saved at: tests/solutions/lrta*/hard_map2/hard_map2_solution.gif

Solution visualization saved in tests/solutions/lrta*/hard_map2
```

Convert it to video py passing the path to the gif file to video.py

```
(.venv) abdi@mac:~/PycharmProjects/AI_PROJECTS$ python video.py tests/solutions/lrta*/hard_map2/hard_map2_solution.gif
Converting tests/solutions/lrta*/hard_map2/hard_map2_solution.gif to tests/solutions/lrta*/hard_map2/hard_map2_solution.mp4 (estimated 1541 frames at 24 FPS)...
Processed frame 1/1541
Processed frame 101/1541
Processed frame 201/1541
Processed frame 301/1541
Processed frame 401/1541
Processed frame 501/1541
Processed frame 601/1541
Processed frame 701/1541
Processed frame 801/1541
Processed frame 901/1541
Processed frame 1001/1541
Processed frame 1101/1541
Processed frame 1201/1541
Processed frame 1301/1541
Processed frame 1401/1541
Processed frame 1501/1541
Successfully converted GIF to tests/solutions/lrta*/hard_map2/hard_map2_solution.mp4
```

7.1.2 Jupyter Notebook

1. Open **main.ipynb**
2. Execute cells **sequentially** to:
 - View **interactive visualizations**
 - See **initial and solved states** for each map
 - Generate **performance comparison plots**:
 - Execution time
 - Move counts
 - Pull move statistics

plot the comparisons

```
1 plot_comparisons(results)
✓ [59] 934ms
```

